> **Thinking 5.1** 如果通过 kseg0 读写设备，那么对于设备的写入会缓存到 Cache 中。这是一种**错误**的行为，在实际编写代码的时候这么做会引发不可预知的问题。请思考：这么做这会引发什么问题？对于不同种类的设备（如我们提到的串口设备和 IDE 磁盘）的操作会有差异吗？可以从缓存的性质和缓存更新的策略来考虑。 ■

访问不同种类的设备时可能会导致错误。kseg0 是存放内核的区域，通过缓存进行访问，但是如果在写入设备时将数据缓存到缓存中，就会导致在访问内核时读取到缓存中的错误数据。对不同种类的设备的操作会有所不同。例如，串口的速度相对于 IDE 磁盘更快，因此在操作串口时需要更频繁地刷新数据。而 IDE 磁盘涉及多个线路的数据冲突问题，操作会更为复杂。

> **Thinking 5.2** 查找代码中的相关定义，试回答一个磁盘块中最多能存储多少个文件控制块？一个目录下最多能有多少个文件？我们的文件系统支持的单个文件最大为多大？ ■

一个磁盘块中最多能存储16个文件控制块。一个目录下最多能有1024*16=16384个文件。

文件系统支持的单个文件最大为1024*4KB = 4MB。

> **Thinking 5.3** 请思考，在满足磁盘块缓存的设计的前提下，我们实验使用的内核支持的最大磁盘大小是多少？ ■

最大磁盘大小是0x6f3fd000。

> **Thinking 5.4** 在本实验中，`fs/serv.h`、`user/include/fs.h` 等文件中出现了许多宏定义，试列举你认为较为重要的宏定义，同时进行解释，并描述其主要应用之处。 ■

#define BLOCK_SIZE PAGE_SIZE

解释：将文件系统块的大小设置为与页面大小相同。

应用：用于确保文件系统块大小与内存页面大小一致。

#define MAXPATHLEN 1024

解释：定义完整路径名的最大长度，包括空字符。

应用：确保路径名不超过此长度，有助于文件路径的导航和管理。

#define FTYPE_REG 0 和 #define FTYPE_DIR 1

解释：定义常规文件和目录的文件类型。

应用：用于区分文件系统中的不同文件类型，以便适当处理每种类型。

> **Thinking 5.5** 在 Lab4“系统调用与 fork”的实验中我们实现了极为重要的 `fork` 函数。那么 fork 前后的父子进程是否会共享文件描述符和定位指针呢？请在完成上述练习的基础上编写一个程序进行验证。 ■

部分关键测试程序如下：

```
int main(void) {
    int fd = open("./test.cap", O_RDWR);
    if (-1 == fd) {
        perror("file open error");
        return -1;
    }

    pid_t pid;
    if ((pid = fork()) < 0) {
        perror("fork error");
    } else if (pid == 0) {
        printf("child₩n");
        printFdInfo(fd);
        printf("child seek %ld₩n", lseek(fd, 3, SEEK_CUR));
        printFdInfo(fd);
    } else {
        sleep(2);
        printf("parent₩n");
        printFdInfo(fd);
        printf("parent seek %ld₩n", lseek(fd, 3, SEEK_CUR));
        printFdInfo(fd);
    }

    close(fd);
    return 0;
}
```

Thinking 5.6 请解释 `File`, `Fd`, `Filefd` 结构体及其各个域的作用。比如各个结构体会在哪些过程中被使用，是否对应磁盘上的物理实体还是单纯的内存数据等。说明形式自定，要求简洁明了，可大致勾勒出文件系统数据结构与物理实体的对应关系与设计框架。 ■

File: 管理文件元数据和数据块指针，对应磁盘上的文件实体。

Fd: 表示进程打开的文件及其状态，管理文件操作位置，不直接对应物理实体。

Filefd: 将文件描述符与 File 结构体关联，用于文件操作。

Thinking 5.7 图 5.9 中有多种不同形式的箭头，请解释这些不同箭头的差别，并思考我们的操作系统是如何实现对应类型的进程间通信的。 ■

黑色实线和黑色实心三角箭头表示同步消息，这意味着消息发送者将进程控制权交给消息接收者，并暂停活动以等待回应。两条小线的开箭头和黑色实线代表异步消息，不需要等待回应。开口三角箭头和黑色虚线表示返回消息，通常与同步消息一起使用。使用开口三角箭头和黑色实线表示创建消息，并在下方特别注明“create”。使用黑色实心三角箭头和黑色实线表示摧毁消息，并在下方特别注明“destroy”。操作系统通过env_ipc_recving来实现进程间的同步。